

NEXUS-R Benchmark Presentation Style

Numbers without methodology are marketing. Methodology without numbers is theory.

Principles

- 1. **Show your work** — Every number must have a reproducible command or script
- 2. **Honest baselines** — Compare against the naive alternative (always-frontier), not against nothing
- 3. **Error bars or variance** — Report p50/p99, not just averages
- 4. **Hardware context** — Specify machine, OS, and load conditions
- 5. **Raw data available** — Link to full benchmark logs, not just summaries

Required Benchmark Sections

1. Environment

```
### Test Environment

| Attribute | Value |
|-----|-----|
| OS | Windows 11 Pro (23H2) |
| CPU | AMD Ryzen 7 5800X (8C/16T) |
| RAM | 32GB DDR4-3200 |
| Storage | Samsung 980 Pro 1TB NVMe |
| Python | 3.12.4 |
| Ollama | 0.1.38 |
| Model | qwen2.5:1.5b-instruct (Q4_K_M) |
| Date | 2026-05-23 |
```

2. Methodology

```
### Methodology

**EventStore Append Test**
- Tool: `scripts/phase1_benchmark.py`
- Command: `python scripts/phase1_benchmark.py --test eventstore --count 10000`
- Metric: Time to append N events to SQLite with causal chaining
- Warmup: 100 events discarded before measurement
- Repetitions: 5 runs, reported value is median
```

3. Results Table

```
### Results

| Metric | Value | Unit | Notes |
|-----|-----|-----|-----|
| Single append (median) | 1.16 | ms | Sequential, no batching |
| Single append (p99) | 2.40 | ms | Worst case under light load |
| Batched append (1000) | 0.024 | ms/event | 1000 events in one transaction |
```

Batched append (10000)	0.023	ms/event	Amortized cost stable
Concurrent writes (10)	3.20	ms/append	Lock contention emerges
Read by ID	0.15	ms	Indexed lookup
Causal chain (100 events)	12.50	ms	Full ancestry traversal

4. Comparison

Comparison

Approach	Latency (1000 events)	Correct?	Notes
-----	-----	-----	-----
NEXUS-R (batched)	24 ms	Yes	WAL mode, connection pool
NEXUS-R (Phase 1, per-write conn)	42,000 ms	Yes	Original implementation
Raw SQLite (no WAL)	35 ms	Yes	Baseline
In-memory list	2 ms	No	Not durable

****Interpretation:**** Batching is 1750x faster than the original per-write connection model and is the correct production path. Single-append at 1.16ms is acceptable for interactive tasks.

5. Limitations

Known Limitations

- Single-append path (1.16ms) exceeds the original <1ms target. Relaxed to <2ms.
- Concurrent writes beyond 10 threads show lock contention. Not yet optimized.
- Tests run on local SSD. Network-attached storage would degrade performance.
- Ollama model is small (1.5B). Larger models (7B, 70B) would increase inference latency proportionally.

6. Reproduction

Reproduction

```
```bash
git clone https://github.com/yourorg/nexus-r.git
cd nexus-r
git checkout ab677aa
pip install -e ".[dev]"
ollama pull qwen2.5:1.5b-instruct
python scripts/phase1_benchmark.py --full
```

Full logs: [benchmark\\_logs\\_20260523.zip](#)

---

## Visualization Style

### Charts

Type	Use For	Tool
-----	-----	-----
Line chart	Latency over time, cost accumulation	Matplotlib / Plotly

```
| Bar chart | Comparison between approaches | Matplotlib |
| Histogram | Latency distribution (p50/p99) | Matplotlib |
| Heatmap | Concurrency vs. throughput | Seaborn |
| Flamegraph | CPU profiling | py-spy |
```

### ### Chart Rules

- **Y-axis starts at 0** unless log scale is explicitly justified
- **Error bars** on every average
- **Raw data points** overlaid when  $N < 50$
- **Annotations** for anomalies ("Spike at 47s: GC pause")
- **Color meaning consistent** across all charts:
  - Green: Local / success / low cost
  - Yellow: BYOK / warning / medium cost
  - Red: Managed premium / failure / high cost
  - Blue: Baseline / comparison

---

## ## Report Template

Use this structure for every benchmark report:

```
```markdown
```

```
# Benchmark Report: [Component] — [Date]
```

Summary

One-paragraph executive summary. Include the single most important number.

Environment

Hardware, software, versions.

Methodology

What was measured, how, and why this metric matters.

Results

Tables and charts.

Comparison

Against baselines, previous versions, or theoretical limits.

Limitations

What this benchmark does NOT prove.

Reproduction

Exact commands to reproduce.

Raw Data

Links to logs, traces, profiles.

Sign-off

Role	Name	Date	Status
-----	-----	-----	-----
Executor	Codex	2026-05-23	Complete
Reviewer	[Human]	[Pending]	[Pending]

Example: Phase 1.5 Report (Condensed)

Benchmark Report: Phase 1.5 Runtime Validation — 2026-05-23

Summary

NEXUS-R Phase 1.5 runtime foundation validated with real Ollama inference.
100 concurrent tasks completed with zero failures. EventStore batch performance at 0.024ms/append. Single-append at 1.16ms (target relaxed from <1ms to <2ms).

Environment

[Table as above]

Methodology

[Scripts and commands as above]

Results

Task Execution

Scenario	Tasks	Success Rate	Avg Latency	Max Latency
-----	-----	-----	-----	-----
Single T1	1	100%	1.9s	2.1s
Batch 20	20	100%	1.9s	12.1s
Concurrent 20	20	100%	2.3s	8.4s
Concurrent 50	50	100%	2.8s	15.2s
Concurrent 100	100	100%	3.4s	22.7s

EventStore

Operation	Count	Total Time	Per-Operation
-----	-----	-----	-----
Batched append	10,000	233.78 ms	0.024 ms

```
| Sequential append | 10,000 | 11,610 ms | 1.16 ms |
| Read by event ID | 1,000 | 152 ms | 0.152 ms |
| Causal chain (deep) | 100 | 1,245 ms | 12.45 ms/chain |
```

Provider

```
| Model | Type | Avg Latency | Cost |
|-----|-----|-----|-----|
| qwen2.5:1.5b-instruct | Local | 1.8s | $0.000 |
| [BYOK placeholder] | Cloud | — | — |
```

Comparison

```
| Metric | Phase 1 (Original) | Phase 1.5 (Now) | Improvement |
|-----|-----|-----|-----|
| EventStore append | 42 ms | 0.024 ms (batched) | 1750x |
| Real model execution | No | Yes | N/A |
| Concurrency tested | 0 | 100 tasks | N/A |
| Telemetry | None | Structured JSON | N/A |
```

Limitations

- BYOK fallback unverified (no live API key)
- Single-append 1.16ms exceeds original <1ms target
- Ollama shutdown mid-flight not yet tested
- 100 concurrent tasks on 1.5B model; larger models untested

Reproduction

```
```bash
git checkout ab677aa
python scripts/phase1_benchmark.py --full
python scripts/phase1_real_validation.py --ollama --model qwen2.5:1.5b-instruct
python tests/stress/test_concurrency_100.py
```

Sign-off

Role	Name	Date	Status
Executor	Codex	2026-05-23	Complete
Reviewer	[Pending]	—	Pending

““

Anti-Patterns (Never Do These)

Anti-Pattern	Why Wrong	Correct Alternative
“99.9% faster”	No baseline specified	“1750x faster than per-write connection model”
Average only	Hides tail latency	Report p50/p99 alongside mean
Best of 5 runs	Cherry-picking	Report median of 5+ runs
No hardware spec	Irreproducible	Full environment table
“Zero latency”	Physically impossible	“<1ms routing overhead”
Missing methodology	Unverifiable	Exact command + script link
No limitations	Dishonest	Explicit known limitations section

A benchmark without reproducibility is a press release. A benchmark without limitations is a lie.